Single Qubit Randomized Benchmarking - Simulation ReadMe

Tanay Bali (Lab Partner: Atal Sahu)

April 2026

1 Code Structure

As explained in the implementation plan write-up, single-qubit randomized benchmarking is implemented in Python using numerous packages and libraries to introduce and simulate practical Clifford gate sequence applications to a prepared initial quantum state. This write-up presents the eight modules that were utilized to construct this simulator.

1.1 Global Parameters

The parameters that were used in the initial simulation are listed in the table below:

Constant	Value
$N_{\max}$	150
K	100
$e_{\max}$	0.05
N_{shots}	1024
SEED	42
μ	0.005
σ	0.001

Table 1: Table of global simulation parameters used in initial simulation runs

Note, this table introduces two parameters, μ, σ, that were not presented in the implementation plan. μ denotes the mean of the Gaussian noise distribution that was used for the error simulation on each gate, and σ denotes the standard deviation of the Gaussian noise distribution.

1.2 Clifford Group Construction

Functions used: `matrix_key(U)`, `clifford_group()`

The single-qubit Clifford group has 24 elements. These are constructed by

using the Identity, I, and the four generators, $H, S, H\dagger, S\dagger$ via a Breadth-First Search (BFS).

`matrix_key(U)`: This function produces a hashable canonical representation of any $2x2$ unitary matrix by removing its global phase. It works by finding the first element of the matrix with a non-negligible magnitude and then dividing the entire matrix by the phase of the chosen element to make it real and positive, rounding it to six figures, and then flattening it to a tuple. This process ensures that any two matrices that represent the same physical gate always produce the same marker key regardless of global phase.

`clifford_group()`: This function runs the BFS by beginning with the Identity, and then applying each of the four generators to every new discovered gate in the Clifford set, and adds any new "key" to the seen dictionary. We provide an assertion that ensures that after 24 elements unique elements are covered in the dictionary, and the Clifford set is returned as a list. This list is stored in the constant, `CLIFFORD_SET`.

1.3 Clifford Gate Labels

Functions used: `clifford_labels()`, `clifford_names(G)`, `find_clifford_inverse(U_cumul)`

Each of the 24 Clifford gates is given human-readable labels (necessary for NMR pulse parsing). The 7 initial single-token names are: I, X, Y, Z, H, S, Sd. For the remainder of the 17 elements, we utilized the Pauli conjugation signature to provide unique labels. For each Pauli axis, (X, Y, Z), the image, $UPU\dagger$ is computed and then matched to one of $\{\pm X, \pm Y, \pm Z\}$. The resulting triplet, for example, $+X + Z - Y$, becomes a unique identifier for every Clifford gate since no two Clifford gates produce the same permutation of the Pauli axis (rigorous since Clifford group is defined to preserve the Pauli group under conjugation). These labels are then stored in `CLIFFORD_LABEL_SET`, and are retrieved by `clifford_names(G)` via the dictionary lookup provided by `matrix_key(G)`.

`find_clifford_inverse(U_cumul)`: This function locates the unqiue element in `CLIFFORD_SET` that equals `U_cumul` upto a global phase by looking up `matrix_key(U_cumul` directly as `U_cumul` is always a product of exact group members drawn from the `CLIFFORD_SET`. The function identifies the inverse gate, `C_inv`, in `CLIFFORD_SET` that satisfies $C_{inv}U_{cumul} \approx \alpha I$ where α is some scalar. This guarantees that an ideal inversion gate is always found rather than a numerically drifted conjugate transpose version.

1.4 Noise Channel

Function used: `depolarize(`ρ`, `q`)`

Each gate has an independent noise value that is applied following a single-

qubit depolarizing channel, which is parametrized by the error probability q:

$$E(\rho) = (1-q)\rho + (q/3)(X\rho X\dagger + Y\rho Y\dagger + Z\rho Z\dagger)$$

This channel is used as the standard depolarizing map from equation (7) of Tripathi et al.

The strength of q is drawn from a clipped Gaussian distribution defined by $N(\mu, \sigma)$ which is bounded below by 0. This is a realistic model that more accurately simulates physical results than other distributions, such as a uniform draw.

1.5 RB Sequence Execution

Function used: `rb_sequence(n, $e_{max}$, $\rho_{in}$)`

This function is the meat of the code, and implements one complete RB sequence of length n, and returns the survival probability along with the ordered list of the gates applied in the randomly generated sequence.

The loop runs n times, and for each iteration a random gate index is drawn, then the corresponding exact Clifford G is retrieved from `CLIFFORD_SET`, its name is added to `gate_names`, and the gate is applied to the density matrix ρ as $G\rho G\dagger$, immediately followed by a noise draw q from the distribution and the depolarizing channel is applied - these two steps model a single noisy gate application matching equation (7) of Tripathi et al. The gate matrix G is also appended to the `gate_sequence`, which is a separate list used to compute the single inversion gate needed to revert each random sequence.

After this loop, `U_cumul` is computed from the stored `gate_sequence` by left multiplying as follows:

$$U_{cumul} = GU_{cumul}$$

for each G in order. This left multiplication ensures that

$$U_{cumul} = G_n \cdot G_{n-1} \cdot ... \cdot G_1$$

which represents the total unitary evolution. The noise-free inversion gate, `C_inv` is then found using `find_clifford_inverse(U_cumul)` and is applied to ρ without noise.

The survival probability is found as

$$P = Tr(|0\rangle\langle 0| \cdot \rho)$$

and is clipped to $[0, 1]$. It is then passed through a binomial distribution (draw) with `N_shots` to simulate measurement statistics. The function returns this survival probability, the list of random gate sequences, and the inversion gate separately in order to log them to a CSV file.

1.6 Data Collection

Functions used: `rb_mod(`n, A, p, B`)`, `fit_rb_func(lengths, `p_{avg}, p_{std}`)`

The RB survival probability is given by

$$P_{avg}(n) = A \cdot p^n + B$$

with p being the decay parameter that hosts the average gate error, and A, B absorb the SPAM errors.

`fit_rb_func`: This function fits the data to this model using `scipy.optimize.curve_fit` with an initial guess $(A, p, B) = (0.5, 0.9, 0.5)$ and with bounds on all parameters to be $[0, 1]$. Runtime errors are implemented to prevent unregulated crashes.

1.7 Gate Infidelity Conversion

Function used: `gate_infidelity_convert(`$p, D = 2$`)`

The decay parameter (fitted), p, is then converted to the average Clifford gate infidelity, r_C, using equation (17) from Tripathi et al. as follows:

$$r_C = (D-1)/D \cdot (1-p) = (1-p)/2$$

for single-qubits. We propagate the 1σ uncertainty in r_C from the covariance matrix of the fit using

$$\sigma(r_C) = \sigma(p)/2$$

1.8 CSV Export

Function used: `export_gate_sequences(sequence_record, path)`

Each gate sequence simulated is written to a CSV file stored at `CSV_path` in order to be retained for the pulse compiler. The CSV has the following columns:

Column	Description
`sequence_length`	Number of random Clifford gates in this run (n)
`run_index`	Which of the K runs for this length
`survival_probability`	Measured survival probability for this run
`inversion_gate`	The single Clifford inversion gate needed to invert the sequence
`gate_1 ... gate_n`	the n random Clifford gates in order of application

Table 2: Table listing the columns and descriptions in the output CSV file

Note, gate names containing $\pm$ characters cause render issues in the CSV file; the underlying data is intact and can still be used by the pulse compiler.

1.9 Visualization

Function used: `plot_rb(lengths, `p_{avg}, p_{std}`, popt, `r_C`, `e_{max}`, save_path`

The output plot has blue/red data points with $\pm 1\sigma$ error bars, which is

the measured $P_{avg}(n)$ at each sequence length. The yellow fit curve is the exponential fit to the data provided by `fit_rb_func(lengths,` p_{avg}, p_{std}`)`. This figure is then saved to `rb_single_qubit.png`.

1.10 Main

Function used: `main()`

The main function coordinates all other functions and allows for parameter changes in order to simulate different physical scenarios in single-qubit RB.

2 Example Simulation Output

For the listed global parameter values stated in this write-up, the following data plot and screenshots of the terminal output/CSV output are shown to understand simulation specifics:

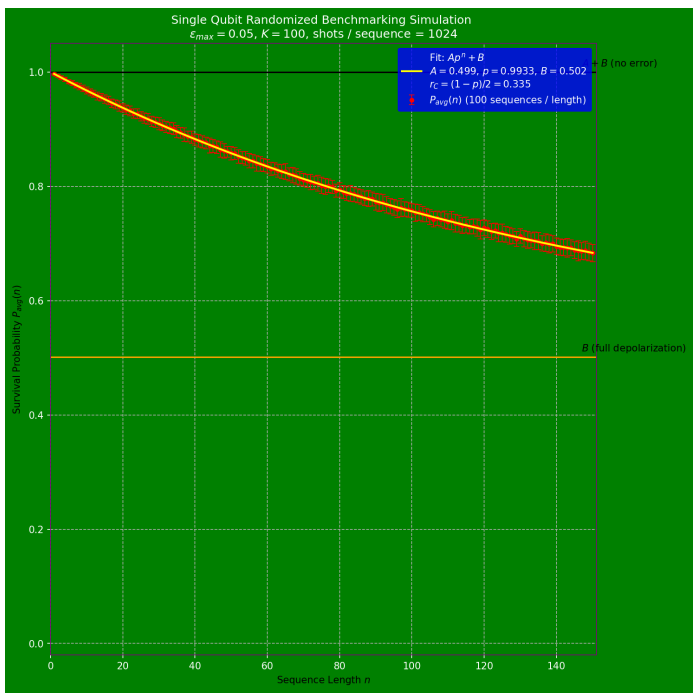

Figure 1: Plot survival probability versus sequence length for RB in single-qubits

```
 [CSV] Written 15000 sequences → rb_sequences.csv  (7584.5 KB)
  [CSV] Columns: sequence_length, run_index, survival_prob, gate_1 … gate_150
  [CSV] Gate_1 … gate_<m> are random Clifford gates; gate_<m+1> is the recovery gate.
Randomized Benchmarking Results
A (SPAM Amplitude) = 0.4986 +- 0.0219
p (decay parameter) = 0.99331 +- 0.00044
B (SPAM Offset) = 0.5016 +- 0.0224
SPAM Error = 1 - (A + B) = -0.0001

 Average Gate Infidelity r_C = (1 - p) / 2
 r_C = 0.3347% +- 0.0219%
```

Figure 2: Python terminal output showing fit function values and gate infidelity value for one set of global parameters

sequence_	run_index	survival_pr	inversion_	gate_1	gate_2	gate_3	gate_4	gate_5	gate_6	gate_7	gate_8	gate_9	gate_10	gate_11	gate_12	gate_13	gate_14	gate_15	gate_16	gate_17	gate_18	gate_19
1	0	0.996094	Sd	S																		
1	1	0.998047	−Y+Z-X	−Z-X+Y																		
1	2	0.995117	−X-Z-Y	−X-Z-Y																		
1	3	0.99707	Y	Y																		
1	4	0.995117	−X-Z-Y	−X-Z-Y																		
1	5	0.996094	−+Z+X+Y	−+Y+Z+X																		
1	6	0.999023	−Z+Y+X	−+Z+Y-X																		
1	7	0.995117	−Y-Z+X	−+Z-X-Y																		
1	8	0.993164	S	Sd																		
1	9	0.995117	−Y+Z-X	−Z-X+Y																		
1	10	0.999023	−Y+Z-X	−Z-X+Y																		
1	11	0.996094	−+Y+Z+X	−+Z+X+Y																		
1	12	0.998047	−Z-Y-X	−Z-Y-X																		
1	13	0.99707	−Z+X-Y	−+Y-Z-X																		
1	14	0.99707	−+Y-Z-X	−Z+X-Y																		
1	15	0.99707	−+Y+Z+X	−+Z+X+Y																		
1	16	0.994141	−Z-Y-X	−Z-Y-X																		
1	17	0.99707	−Z+Y+X	−+Z+Y-X																		
1	18	0.998047	−Y-X-Z	−Y-X-Z																		
1	19	1	−+Z-X-Y	−+Y-Z+X																		
1	20	0.995117	−+Y-Z-X	−Z+X-Y																		
1	21	0.99707	−Y+Z-X	−Z-X+Y																		
1	22	0.999023	−X-Z-Y	−X-Z-Y																		
1	23	0.99707	−+Z+Y-X	−Z+Y+X																		
1	24	0.996094	−+Y+X-Z	−+Y+X-Z																		

Figure 3: CSV output showing random gate sequences values and survival probability values